



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: (Summer, Year: 2025), B.Sc. in CSE (EVE)*

ByteTalk: A Real-Time Chat Application

*Course Title: Computer Networking Lab
Course Code: CSE 312
Section: 223 E2*

Students Details

Name	ID
Md Sium	223015036
Bijoy Basak	223015032

*Submission Date: 02.09.2025
Course Teacher's Name: Md. Zahidul Hasan*

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	2
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition	2
1.3.1	Problem Statement	2
1.3.2	Complex Engineering Problem	3
1.4	Design Goals/Objectives	3
1.5	Application	3
2	Design/Development/Implementation of the Project	4
2.1	Introduction	4
2.2	Project Details	4
2.3	The Key Features	5
2.4	Tools and Libraries	5
2.5	The Workflow	6
2.6	Code Section	7
3	Performance Evaluation	9
3.1	Simulation Environment/Simulation Procedure	9
3.2	Results Analysis/Testing	10
4	Conclusion	12
4.1	Discussion	12
4.2	Limitations	12
4.3	Scope for Future Work	12
5	References	14

Chapter 1

Introduction

1.1 Overview

In the contemporary digital landscape, communication has become an essential element in bridging the gap between individuals and organizations globally. Despite the abundance of chat applications available, many fall short in providing an intuitive and comprehensive platform that facilitates seamless message, file, and image exchange. Our project, titled "ByteTalk" aims to develop a simple yet robust chat application using Java Socket Programming and Swing GUI. This application not only supports real-time text communication but also integrates file and image transfer functionalities, thereby enhancing the user experience and meeting the diverse communication needs of users.

1.2 Motivation

The primary motivations behind the development of "ByteTalk" are centered around simplifying communication, enhancing collaboration:

- **Simplifying Communication:** Many existing chat applications are overly complex. Our project aims to simplify the process of exchanging messages, files, and images, making it accessible to users of all technical backgrounds.
- **Enhancing Collaboration:** Effective collaboration often requires swift and efficient communication coupled with seamless sharing of files and images. Our application seeks to facilitate collaboration by providing an integrated platform.

1.3 Problem Definition

1.3.1 Problem Statement

The primary challenge of this project is to develop a real-time chat application that can handle multiple users, facilitate the secure exchange of messages and files, and provide a responsive and user-friendly graphical interface.

1.3.2 Complex Engineering Problem

Problem Aspect	Explanation
Depth of knowledge required	Java Socket programming, Java Swing, multi-threading, file and image handling.
Conflicting requirements	Real-time low-latency messaging vs robust error handling.
Depth of analysis required	Client-server architecture with sockets, GUI, threading, file transfer.
Familiarity of issues	Authentication, emoji support, stability, data integrity.

1.4 Design Goals/Objectives

The primary objectives of the *ByteTalk* project are as follows:

- **Real-Time Messaging:** Deliver messages with minimal latency to ensure smooth, immediate communication.
- **Secure File Transfer:** Support sending/receiving files and images while preserving data integrity and privacy.
- **User-Friendly Interface:** Provide a clean, intuitive Java Swing UI for easy navigation and accessibility.
- **Robust Error Handling:** Detect, report, and recover gracefully from failures (e.g., network drops, invalid inputs).
- **Scalability:** Design the architecture to support more users/messages without degrading performance.

1.5 Application

The Chat Application developed in this project, *ByteTalk*, addresses various needs in personal, professional, and educational settings through its comprehensive features such as real-time messaging, file and image transfer, and a user-friendly interface.

- **Professional Collaboration:** In professional environments, the application enhances productivity and teamwork by enabling team members to exchange messages, share files, and coordinate tasks securely.
- **Educational Institutions:** Teachers and students can use “ByteTalk” for project discussions, group studies, and academic collaborations, facilitating the sharing of study materials and feedback.
- **Community Building:** The application can serve as a platform for building and maintaining online communities, supporting meaningful discussions, knowledge sharing, and engagement.

Chapter 2

Design/Development/Implementation of the Project

2.1 Introduction

The *ByteTalk* project aims to develop a comprehensive chat application using Java socket programming and the Swing GUI toolkit. This chapter outlines the project details, workflow, and the tools and libraries used in its development. The focus is on creating a reliable, user-friendly platform that supports real-time messaging, file and image transfer, and secure data handling.

2.2 Project Details

The *ByteTalk* application encompasses several key components designed to facilitate effective communication and collaboration. The main features include:

- **Real-Time Messaging:** Enabling users to send and receive messages instantly.
- **File and Image Transfer:** Allowing users to upload, download, and preview files and images securely.
- **User Authentication:** Ensuring secure login and access control for all users.
- **User-Friendly Interface:** Providing an intuitive and responsive graphical user interface using Java Swing.
- **Data Security:** Implementing encryption techniques to protect data transmission and user information.

The application is designed to be scalable, allowing it to handle an increasing number of users and messages without compromising performance.

2.3 The Key Features

The *ByteTalk* project includes the following features:

1. **User Registration:** New users can create an account by providing necessary details, which are stored securely in the database.
2. **User Authentication:** Users must log in with their credentials, which are verified against the database.
3. **Establishing Connection:** Upon successful login, a socket connection is established between the client and the server.
4. **Real-Time Messaging:** Users can send and receive messages instantly, with messages being relayed through the server.
5. **File and Image Transfer:** Users can upload files and images, which are encrypted and sent to the recipient through the server.
6. **User Logout:** Users can log out, which terminates the socket connection and updates their status to offline.

This workflow ensures a seamless and secure communication experience for all users, providing a comprehensive set of features for effective communication and collaboration.

2.4 Tools and Libraries

The development of *ByteTalk* utilizes various tools and libraries to achieve its objectives:

- **Java Programming Language:** Used for its robustness and versatility in building network-based applications.
- **Java Socket Programming:** Provides the foundation for establishing and managing communication channels between clients and the server.
- **Java Swing GUI Toolkit:** Used for designing a responsive and user-friendly graphical user interface.
- **Java Standard Library:** Utilized for file handling, threading, exception handling, and other core functionalities.
- **IntelliJ IDEA IDE:** Used as the main development environment for coding, debugging, and testing the application.
- **Git:** Version control system for managing source code changes and enabling team collaboration.

- **MySQL:** Database management system for storing user credentials, chat history, and file metadata securely.

These tools and libraries collectively enable the development of a robust and efficient chat application, ensuring a high-quality and secure user experience in *ByteTalk*.

2.5 The Workflow

The workflow of the *ByteTalk* application ensures smooth communication and collaboration between users through the following steps:

1. **Server Setup:** The server application is started and listens on a specified port for incoming client connections. It manages all communication between clients.
2. **User Registration:** New users can create an account by providing the required details. These details are securely stored in the database.
3. **Client Login:** Existing users log in using their credentials, which are verified against the stored database records. Upon successful login, the client interface is initialized.
4. **Establishing Connection:** Once login is successful, a socket connection is established between the client and the server, enabling real-time communication.
5. **Chat Interface:** After login, users can send text messages, attach images, and use an emoji picker for expressive communication.
6. **Message Transmission:** Messages are sent to the server, which relays them to the intended recipient(s). In group chats, the message is distributed to all members.
7. **File and Image Transfer:** Files and images are securely uploaded by the sender, encrypted, and delivered to the recipient through the server.
8. **Server Management:** The server manages active client connections, distributes messages, handles client disconnections, and ensures the reliability of communication.
9. **Server Display:** The server interface displays client activity, including message logs and connection status.
10. **Closing Connections:** When users log out, their socket connections are closed and their status is updated to offline.
11. **UI Responsiveness:** Java Swing ensures that the user interface remains responsive while background processes (like message delivery and file transfer) run smoothly.

This workflow guarantees a reliable, secure, and user-friendly communication experience for all users of *ByteTalk*.

2.6 Code Section

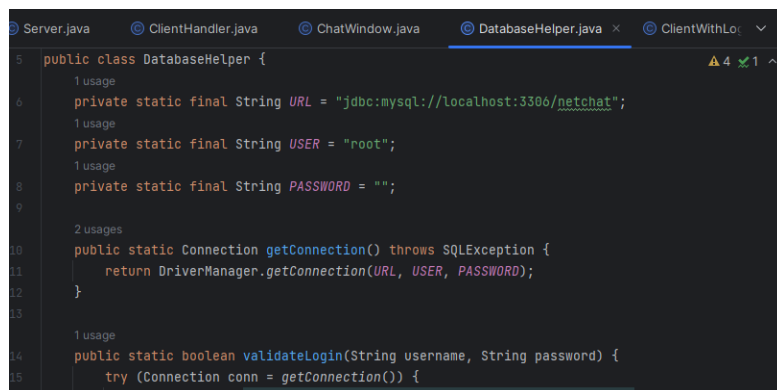
Due to the extensive nature of the code files involved in the *ByteTalk* application, it is not feasible to include all source code within this report. Instead, this section provides an overview of the basic workflow of the system, from server initialization to client login and message transmission.

The server-side implementation begins by initializing the server socket and listening for incoming client connections. When a client attempts to connect, the server accepts the request and creates a new thread to handle client-specific operations. This multi-threaded approach enables the server to manage multiple clients concurrently.

Once a connection is established, the client can send a login request. The server validates the credentials against the stored database records. If the credentials are correct, the server sends a confirmation response, and the client transitions to the chat interface.

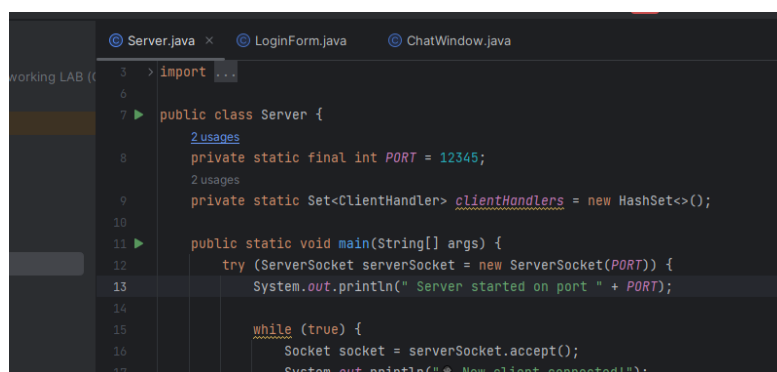
After successful login, clients can exchange messages in real-time. The client encodes the message and transmits it via the socket connection to the server. The server then decodes the message and relays it to the intended recipient(s). A similar process is applied to file and image transfers, ensuring that the transmitted data is delivered securely and displayed correctly on the recipient's interface.

This structured implementation ensures a smooth, reliable, and secure communication experience within the *ByteTalk* application.

A screenshot of an IDE showing the DatabaseHelper.java file. The code defines a class with static fields for database URL, user, and password, and methods for getting a database connection and validating login credentials.

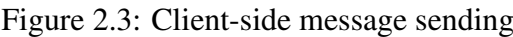
```
1 public class DatabaseHelper {  
2     1 usage  
3     private static final String URL = "jdbc:mysql://localhost:3306/netchat";  
4     1 usage  
5     private static final String USER = "root";  
6     1 usage  
7     private static final String PASSWORD = "";  
8  
9  
10    2 usages  
11    public static Connection getConnection() throws SQLException {  
12        return DriverManager.getConnection(URL, USER, PASSWORD);  
13    }  
14  
15    1 usage  
16    public static boolean validateLogin(String username, String password) {  
17        try (Connection conn = getConnection()) {  
18            Statement stmt = conn.createStatement();  
19            String query = "SELECT * FROM users WHERE username = ? AND password = ?";  
20            PreparedStatement pstmt = conn.prepareStatement(query);  
21            pstmt.setString(1, username);  
22            pstmt.setString(2, password);  
23            ResultSet rs = pstmt.executeQuery();  
24            return rs.next();  
25        }  
26    }  
27 }
```

Figure 2.1: Connection with Database

A screenshot of an IDE showing the Server.java file. The code defines a Server class with a main method that initializes a server socket and listens for incoming connections.

```
1 > import java.net.*;  
2  
3  
4  
5  
6  
7 public class Server {  
8     2 usages  
9     private static final int PORT = 12345;  
10    2 usages  
11    private static Set<ClientHandler> clientHandlers = new HashSet<>();  
12  
13    public static void main(String[] args) {  
14        try (ServerSocket serverSocket = new ServerSocket(PORT)) {  
15            System.out.println(" Server started on port " + PORT);  
16  
17            while (true) {  
18                Socket socket = serverSocket.accept();  
19                System.out.println(" * New client connected!");  
20            }  
21        }  
22    }  
23 }
```

Figure 2.2: Server-side connection handling



Chapter 3

Performance Evaluation

3.1 Simulation Environment/Simulation Procedure

The *ByteTalk* application was tested in a controlled simulation environment to evaluate its functionality, reliability, and performance. The testing setup included the following:

- **Local Server Hosted on a Linux-based machine** with the necessary database configured to store user credentials, and file metadata.
- **Multiple Clients** Test clients were initiated on Windows systems.
- **Network Configurations** Both Local Area Network (LAN) and Wide Area Network (WAN) scenarios were simulated to test connectivity and latency performance under different conditions.

The simulation procedure was carried out in the following steps:

1. **Server Initialization:** The server was started, initializing the socket and listening for incoming client connections.
2. **Client Connection:** Multiple clients connected to the server, undergoing either the login process (for existing users) or registration (for new users).
3. **Message Exchange:** Clients exchanged text messages, images, and files to test reliability, speed, and correctness of message delivery.
4. **Error Testing:** Various error scenarios were introduced, such as abrupt client disconnections, invalid login attempts, and simulated network failures. The application's error-handling mechanisms were observed and recorded.
5. **Performance Observation:** The server's ability to handle multiple concurrent connections and maintain stability was tested under load.

3.2 Results Analysis/Testing

The testing and analysis of the ByteTalk application yielded the following results:

- **Connection Stability:** The server successfully handled multiple client connections simultaneously without major performance degradation. The application maintained stable communication across different network conditions.
- **Message Delivery:** Text messages, files, and images were delivered reliably with minimal latency. The system proved to be robust and efficient in real-time communication.
- **User Authentication:** The login and registration features functioned correctly, ensuring that only valid users could access the system.
- **Error Handling:** The application responded effectively to simulated failures, including invalid inputs and unexpected disconnections, by providing appropriate feedback and recovering gracefully.

The following figures illustrate parts of the testing results:

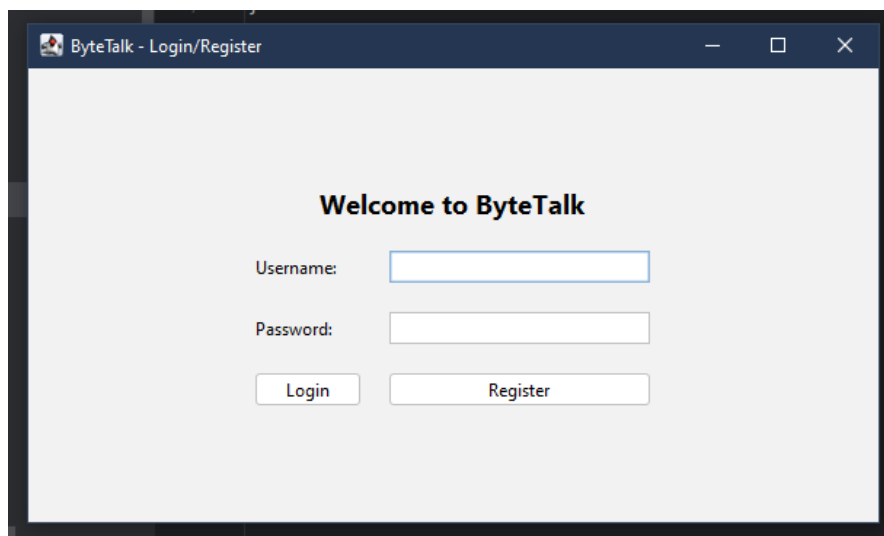


Figure 3.1: Login Screen in ByteTalk

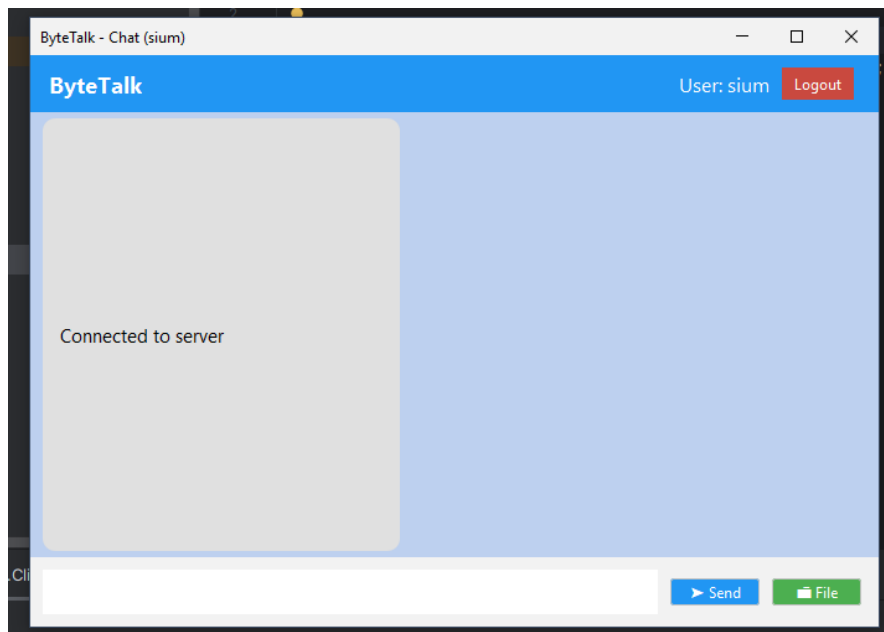


Figure 3.2: Chat Interface with Text Message

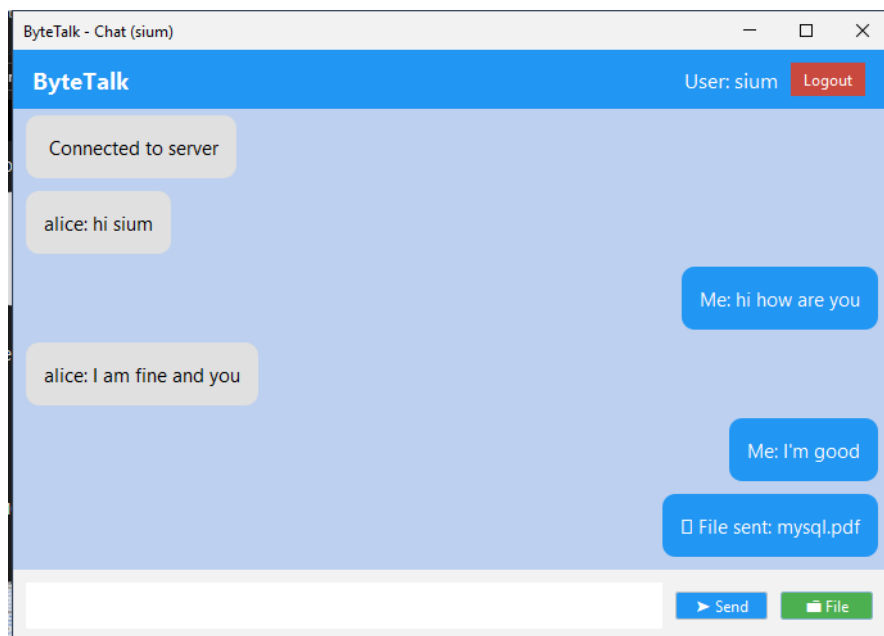


Figure 3.3: Chat Interface with Text Message

Chapter 4

Conclusion

4.1 Discussion

The *ByteTalk* application was designed and implemented to provide a robust, user-friendly, and efficient chat platform. By integrating both server-side and client-side components, it ensures smooth communication between users. The system supports core functionalities such as user authentication, real-time messaging, file transfer, and emoji support. Through socket programming and a Swing-based graphical interface, *ByteTalk* delivers a reliable environment for personal, academic, and professional communication.

4.2 Limitations

Despite its successful implementation, the current version of *ByteTalk* has some limitations that affect performance and usability:

- **Scalability:** The system is not optimized for handling a large number of concurrent users, leading to potential performance issues under heavy loads.
- **Error Handling:** Limited mechanisms exist for handling unexpected failures, which can sometimes cause instability.
- **Cross-Platform Compatibility:** While functional across platforms, performance inconsistencies may arise between different operating systems.
- **Documentation:** The project lacks detailed developer documentation, making future contributions and maintenance more difficult.

4.3 Scope for Future Work

Several improvements can be made to enhance the functionality, performance, and user experience of *ByteTalk*:

- **Improved GUI:** Redesign the interface to be more modern, responsive, and user-friendly.
- **Enhanced File Transfer:** Optimize the transfer mechanism to support larger files with faster and more reliable delivery.
- **Video and Audio Calls:** Extend the platform to include real-time audio and video communication.
- **Advanced Security:** Implement end-to-end encryption, two-factor authentication, and stronger data protection techniques.
- **Scalability Enhancements:** Refactor the server architecture to handle thousands of concurrent users without performance degradation.
- **Cross-Platform Expansion:** Extend support to mobile platforms (Android/iOS) to increase accessibility.

In conclusion, *ByteTalk* currently provides a functional and reliable chat platform. By addressing its limitations and implementing the suggested improvements, the application can evolve into a more powerful, secure, and versatile communication tool capable of serving a wide range of user needs in real-world environments.

Chapter 5

References

1. Google Developers – Education: <https://developers.google.com/edu/python>
2. DigitalOcean – Java Socket Programming (Server–Client): <https://www.digitalocean.com/community/tutorials/java-socket-programming-server-client>
3. Cybercademy – The Socket Programming Project: <https://cybercademy.org/the-socket-programming-project/>